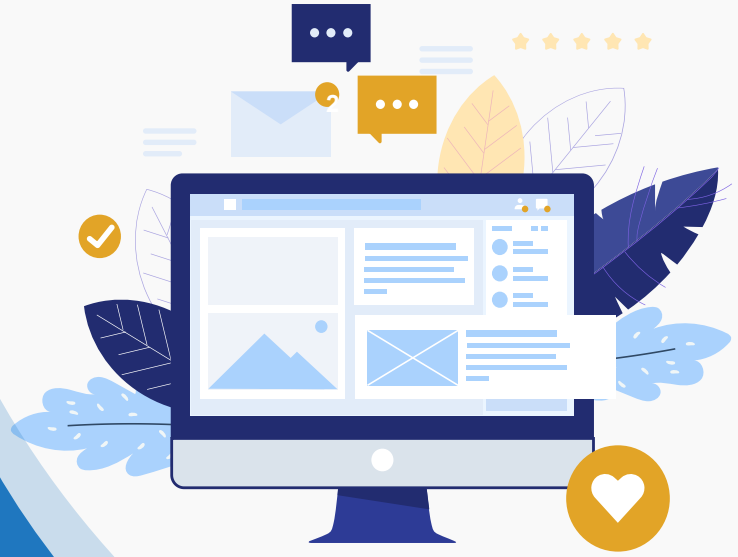
计算机与网络空间安全学院
School of Computer and Cyber Sciences



数据分析与可视化

主讲人：张伟娜

计算机与网络空间安全学院

PART 01

为什么要学习数据分析

为什么要学习数据分析

- 2020 年 4 月（《中共中央 国务院关于构建更加完善的要素市场化配置体制机制的意见》）：正式确立 “土地、劳动力、资本、技术、数据” 五大要素，**数据成为第五生产要素**
- 2025 年 12 月 30 日 全国数据工作会议（国家发改委、国家数据局联合召开）官方首次正式将2026 年确立为 “**数据要素价值释放年**”，作为 2026 年数据工作的核心定位
- 数据在专业领域的作用
  - 投资方用数据评估项目
  - 运营方用数据精准投放、提升播放与变现
  - 平台用数据判断内容热度
  - ……

PART 02

数据分析基本流程

数据分析基本流程

01 明确目标

02 获取数据

03 数据处理

04 数据分析

05 结果展现

数据分析工具

数据分析工具有：Microsoft Excel、 Statistical Analysis System（SAS）、Statistical Package for the Social Sciences（SPSS）、Tableau、R语言、Python语言、大语言模型等

大语言模型进行数据分析的优势：

- 极低的交互门槛（自然语言交互）
- 处理非结构化数据的能力
- 强大的预测与模拟能力
- 自动化治理与清洗

大语言模型进行数据分析的劣势：

- “幻觉”与不可解释性（黑盒效应）
- 数据隐私与安全风险
- 逻辑推理的深度限制
- 对高质量数据依赖

最佳实践是“**人机协同**”。利用 AI 进行初步的探索性分析（Exploratory Data Analysis）和代码编写，人工负责最后的审核和基于行业经验的逻辑验证。

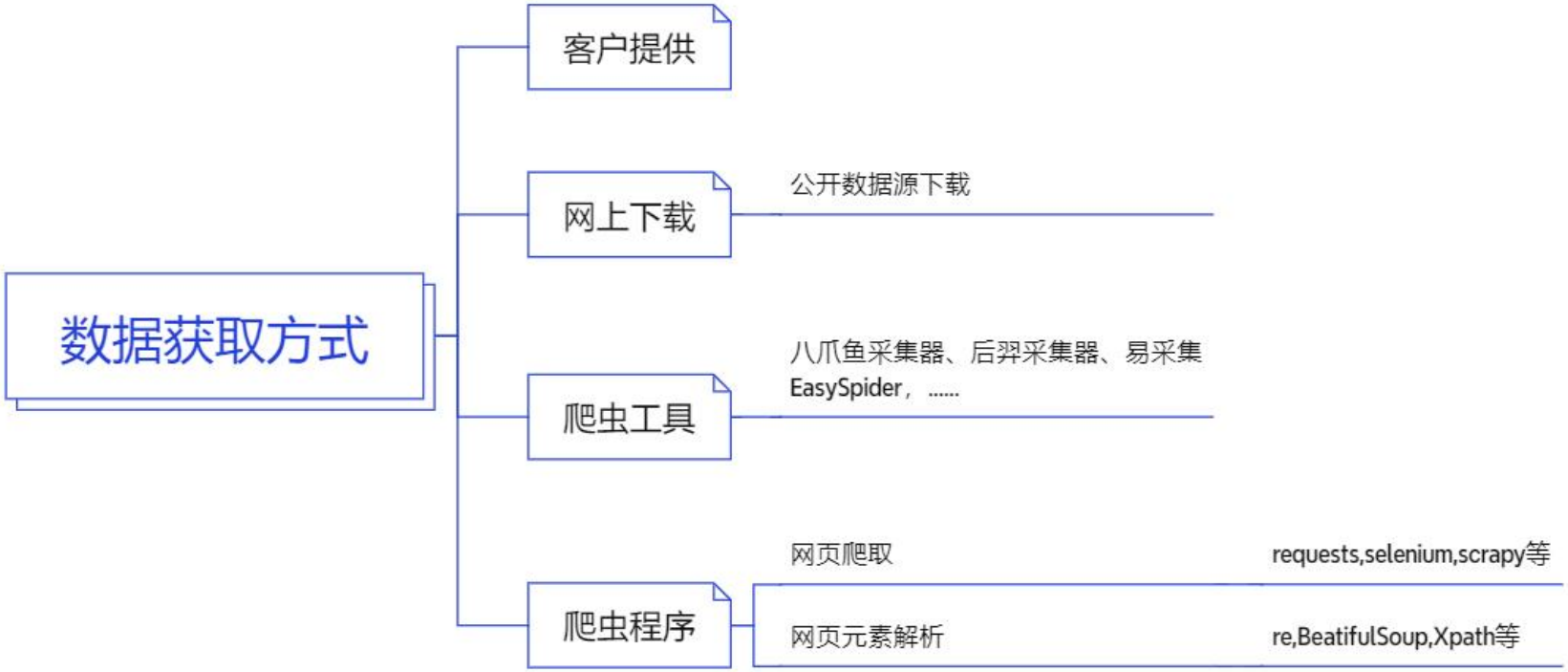
PART 03

数据获取方式

数据获取方式

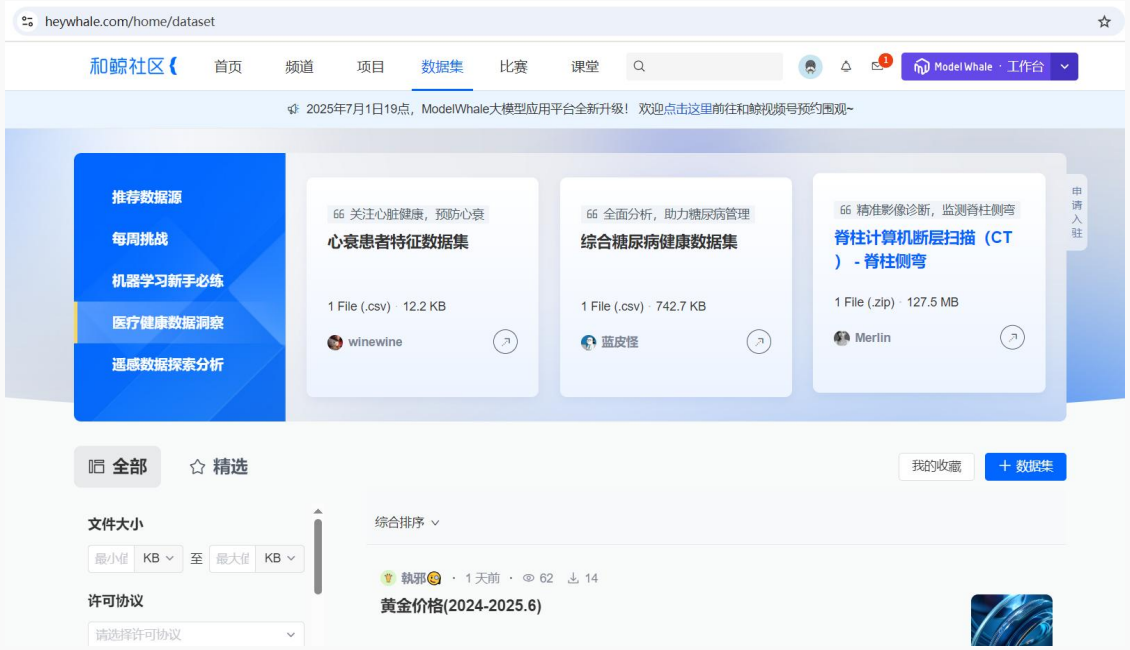
数据与算法就如同工业时代的煤炭与蒸汽机一样，
数据是人工智能时代的新能源。

数据获取方式


数据获取方式
客户提供
网上下载
公开数据源下载
爬虫工具
八爪鱼采集器、后羿采集器、易采集
EasySpider，.....
爬虫程序
网页爬取
requests,selenium,scrapy等
网页元素解析
re,BeatifulSoup,Xpath等

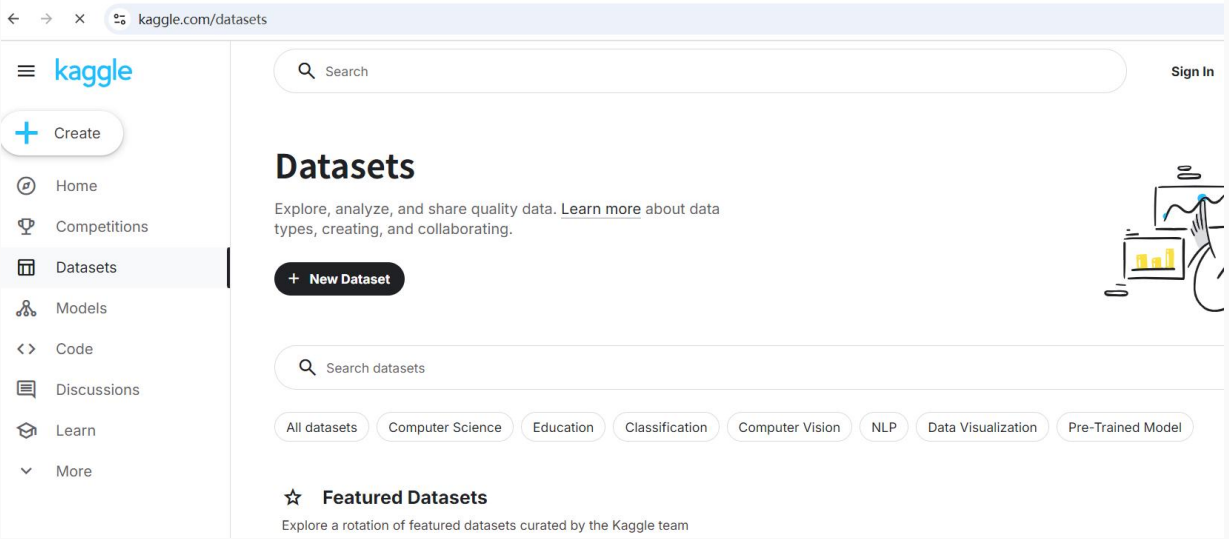
数据获取方式

数据获取方式——公开数据源



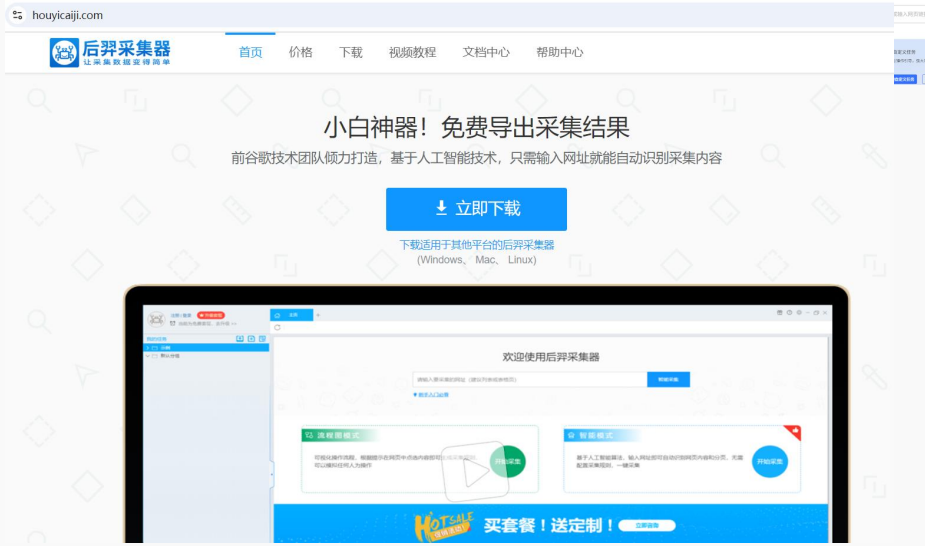
数据获取方式

数据获取方式——公开数据源



数据获取方式

数据获取方式——爬虫工具



PART 04

数据采集规则

数据采集规则

不爬取受版权保护的数据，要避免收集和利用个人敏感信息，如身份证号、银行卡号、医疗记录等。

遵守网站服务条款中的爬取协议，不访问禁止抓取的页面。

不能通过爬虫技术干扰网站正常运营，爬取的数据应用于合法、正当的目的，不能用于恶意用途，如诈骗、网络攻击等。

案例

- 景点分析
- 热门旅游目的地识别
- 热门景点分布

- 评分分布
- 热门景点主题分析

- 价格区间分布
- 价格与销量

PART 05

数据分析

文件读写

Pandas读写文件常用函数

文件格式	读	写
csv	pd.read_csv()	df.to_csv()
excel	pd.read_excel()	df.to_excel()
Python Pickle 格式	pd.read_pickle()	df.to_pickle()
HTML	pd.read_html()	df.to_html()
JSON	pd.read_json()	df.to_json()
……	……	……

文件路径

当前工作目录为：D:/chapter6
需读取“D:/chapter6/data/c.txt”文件，如何描述文件路径？

绝对路径

从文件系统的根目录开始，完整描述文件位置的路径。

如：D:/chapter6/data/c.txt

相对路径

以当前工作目录为参照的路径，描述时省略与当前目录相同的部分。

如：data/c.txt

文件路径

绝对路径

- **特点**
  - ◆唯一性：每个文件的绝对路径在系统中是唯一的。
  - ◆以根目录为起点：不同操作系统的根目录表示方式不同：

  Windows：以盘符开头（如： D:\chapter6\data\c.txt）。

  Linux/macOS：以 / 开头（如： /home/user/chatper6/data/c.txt）。
- **优点：** 明确无歧义，可直接定位文件。
- **缺点：** 路径较长，如果文件目录结构变化（如移动根目录），路径会失效。因此在文件读写时不推荐使用绝对路径。

文件路径

相对路径

D:/chapter6/data/c.txt
D:/chapter6/a.py

data/c.txt

- **特点：**
  - 灵活性：路径更简短，但依赖当前目录环境。
  - 相对路径表示中的符号：
    - . 表示当前目录（例如 ./file.txt）。
    - .. 表示上级目录（例如 ../images/photo.jpg）。
- **优点：** 简洁，适合在项目内部引用文件。
- **缺点：** 必须明确当前目录，否则可能找不到文件。

在进行项目开发时，推荐使用相对路径，以便项目移植和打包分发。

文件路径

例：

在a.py中读取b.txt文件，相对路径为：

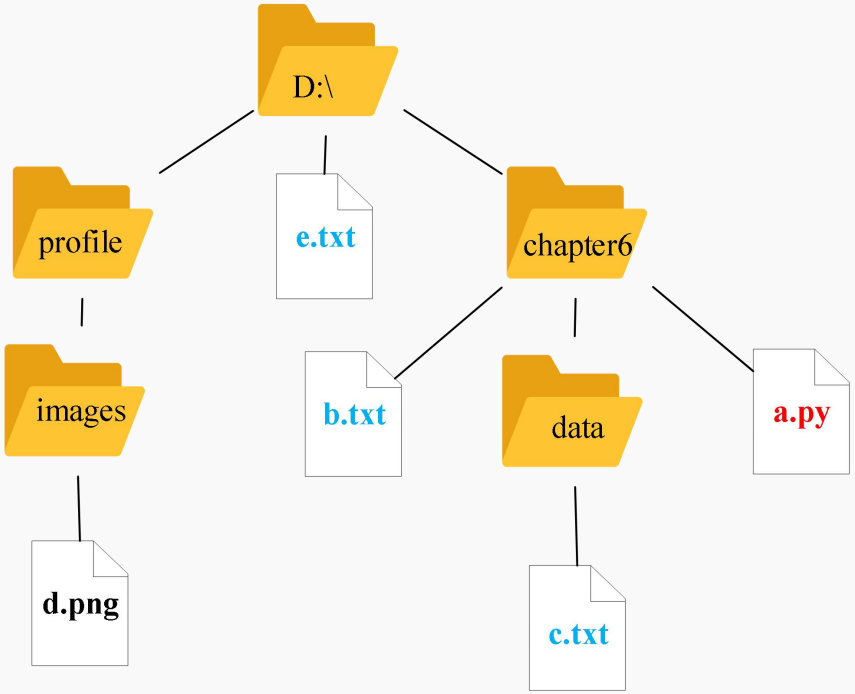
b.txt

在a.py中读取c.txt文件，相对路径为：

data/c.txt

在a.py中读取e.txt文件，相对路径为：

../e.txt



数据基本信息查看

方法	说明
df.head(n)	df开始n行数据，默认n=5
df.tail(n)	df最后n行数据，默认n=5
df.info()	df的简要说明（如索引、列数、列名、数据量、数据类型、每列非空数据个数、内存使用情况等）
df.describe()	对于数字列，describe()返回基本统计信息，如：列中数据的值计数、平均值、标准差、最小值、最大值以及第 25、第 50和第75分位数； 对于字符串列，describe() 返回值计数、唯一条目数、最常出现的值(top value) 以及最高值出现的次数(freq)。

数据基本信息查看

例

读取数据 “旅游景点.xlsx”

查看是否有空缺值

数据处理

- 数据处理
  - 数据清洗
    - 缺失值处理
      - 填充 fillna()
      - 删除 dropna()
    - 重复值处理
      - 删除 drop_duplicates()
    - 异常值处理
      - 删除 drop()
      - 研究异常值
  - 数据加工
    - 数据抽取
    - 数据转换

数据处理-数据清洗

数据清洗

方法	说明
fillna(value)	用value填充缺失值
dropna(how='any/all')	删除包含缺失值的行; how='any'，表示该行/列只要存在缺失值就删除; how='all'，表示该行/列全部为缺失值时才删除。
drop_duplicates()	删除重复出现的值

数据处理-数据清洗

例：

将“星级”中的空缺值用“未知”填充

删除销量为0的数据记录

数据抽取

数据抽取

- 数据抽取
  - 抽取1行
    - df.loc[行名]
    - df.iloc[行索引]
  - 抽取1列
    - df.loc[:,列名]
    - df.iloc[:,列索引]
    - df[列名]
  - 抽取指定行列
    - df.loc[行名,列名]
    - df.iloc[行索引,列索引]
  - 抽取多行多列
    - df.loc[行名1:行名2，列名1：列名2]
    - df.iloc[行索引1：行索引2，列索引1：列索引2]
  - 按条件抽取

数据抽取

例：

读取score.csv文件

（1）查看“张三”的成绩

（2）查看“Python”成绩

（3）查看“李四”的“广告学概论”成绩

（4）查看“李四,小华，小明”的“高数”、“广告学概论”成绩

（5）查看“Python”>90的同学成绩

（6）查看“Python”>90的同学的英语成绩

（7）查看“Python”>90并且“思政”>85的同学成绩

（8）查看“Python”>90 或者“思政”>85的同学成绩

（9）查看“Python”>90 或者“思政”>85的同学“广告学概论”成绩

数据统计分析

常用统计函数

函数	说明
DataFrame.sum([axis,skipna,level,...])	求和
DataFrame.mean([axis,skipna,level,...])	求均值
DataFrame.max([axis,skipna,level,...])	求最大值
DataFrame.min([axis,skipna,level,...])	求最小值
DataFrame.median([axis,skipna,level,...])	求中位数
DataFrame.mode([axis,numeric_only=False,dropna=False])	求众数
DataFrame.var([axis,skipna,level,...])	求方差
DataFrame.std([axis,skipna,level,...])	求标准差
DataFrame.quantile([q=0.5,axis=0,numerical_only=True,interpolation='linear'])	求分位数
……	……

数据统计分析

DataFrame.sum([axis,skipna,level,...])

sum()实现行/列数据的求和运算。

- axis: 表示要求和的轴。axis=0表示逐行；axis=1:表示逐列，默认是逐行。
- skipna: 表示对NaN值的处理方式。skipna=1表示将NaN值转换为0；skipna=0表示NaN值不自动转换。默认NaN值自动转换为0。
- level：表示索引层级。

axis=1

	Python	英语	高数	思政	广告学概论	统计及数据分析基础
张三	95	85	96	88	78	90
李四	85	86	90	70	88	85
小华	80	90	75	85	98	95
小明	78	92	85	72	95	75
小兰	99	88	65	80	85	75

axis=0

数据统计分析

例

- 读取score.csv，并完成下列操作
  - 计算各同学的总成绩，放在“总成绩”列
  - 计算各科的平均分，放在”平均分”行

排序与排名

- 排序与排名
  - 排序
    - 按值排序 df.sort_values()
    - 按行列名称排序 df.sort_index()
  - 排名
    - df.rank()

数据排序与排名

按行或列的值进行排序

```
df.sort_values(by,axis=0,ascending=True,inplace=False,kind='quiksort',na_position='last',ignore_index=False)
```

参数	说明
by	指定排序依据的名称列表
axis	指定排序方向，0：行方向排序; 1:列方向排序
ascending	指定排序顺序，True:升序; False:降序；默认为升序排序
inplace	是否在原对象上进行排序。默认为False，不修改原对象，返回新的DataFrame对象。
kind	指定排序算法。quicksort：快速排序；mergesort：混合排序; heapsort：堆排序；默认为‘quicksort’
na_position	控制缺失值的位置。‘first’：缺失值放在排序结果的开头；‘last’：缺失值放在排序结果的末尾，默认值 为‘last’
ignore_index	排序后是否重置索引。True:标记索引；False：忽略索引

数据排序与排名

按行或列名称进行排序

```
df.sort_index(axis=0, ascending=True, inplace=False, na_position='last', ……)
```

参数含义与sort_values()中的参数相同。

数据排序与排名

排名

```
df.rank(axis=0, method='average', ……)
```

参数	说明
axis	指定计算排名的方向。0：按行排名; 1:按列排名
method	指定如何处理相同值的排名。 average： 相同值取平均排名（默认） min： 相同值取最小排名 max： 相同值取最大排名 first： 按值在原始数据中的出现顺序依次分配排名

数据排序与排名

例：

读取bookSales.xlsx

(1) 按“销量”降序排序

(2) 按“销量”和“图书名称”降序排序

(3) 按列名进行升序排列

(4) 对销量进行排名

思考

数据如右表所示

- 计算各公司的总销售额
- 计算各类商品的平均单价
- 计算各公司的总销售额以及平均销售量
- 计算各公司各类商品的总销售额

日期	商品类别	商品名称	公司	销售量	单价	销售额
2019/11/11	手机	Apple iPhone 11	京东	10	5999	59990
2019/11/11	手机	Apple iPhone XR	京东	16	5099	81584
2019/11/11	数码相机	佳能（Canon）EOS 80D	京东	10	8299	82990
2019/11/11	数码相机	佳能（Canon）EOS 5D Mark IV	京东	5	16299	81495
2019/11/11	笔记本	DELLXPS13.3英寸	京东	20	9399	187980
2019/11/11	笔记本	华为笔记本 MateBook 14	京东	22	5699	125378
2019/11/11	笔记本	Apple Macbook Air 13.3	京东	18	5999	107982
2019/11/11	移动硬盘	三星(SAMSUNG) 500GB 固态硬盘	京东	20	669	13380
2019/11/11	手机	华为 HUAWEI Mate 30	京东	18	4299	77382
2019/11/11	手机	小米9 Pro	京东	18	3799	68382
2019/11/11	数码相机	佳能（Canon）EOS 80D	拼多多	6	8299	49794
2019/11/11	数码相机	佳能（Canon）EOS 5D Mark IV	拼多多	3	16299	48897
2019/11/11	数码相机	尼康（Nikon）D750	拼多多	8	11899	95192
2019/11/11	笔记本	Apple Macbook Pro 13.3	拼多多	6	10699	64194
2019/11/11	手机	华为 HUAWEI Mate 30 Pro	苏宁易购	8	6299	50392
2019/11/11	笔记本	ThinkPad T480	苏宁易购	12	10999	131988
2019/11/11	笔记本	ThinkPad X1 Carbon	苏宁易购	12	14999	179988
2019/11/11	移动硬盘	希捷(Seagate)2TB	苏宁易购	30	499	14970
2019/11/11	移动硬盘	希捷(Seagate)4TB	苏宁易购	12	789	9468
2019/11/11	手机	Apple iPhone 11	天猫	30	5999	179970
2019/11/11	手机	Apple iPhone XR	天猫	40	5099	203960
2019/11/11	移动硬盘	西部数据(WD)2TB	天猫	20	489	9780
2019/11/11	移动硬盘	西部数据(WD)4TB	天猫	30	799	23970
2019/11/11	数码相机	佳能（Canon）EOS 800D	天猫	5	4999	24995
2019/11/11	数码相机	尼康（Nikon）D850	天猫	4	18299	73196

分类汇总

- 数据汇总
  - 分类汇总
    - 分类汇总是对相同类别的数据进行统计汇总。即先对数据进行类别划分，然后对同类别的数据进行求和、求平均、计数等汇总运算。
    - 汇总方法
      - df.groupby(分类列名）["列名"].汇总函数
      - df.groupby(分类列名）.agg({"列名"：汇总函数，"列名":"汇总函数"})
  - 数据透视表
    - 数据透视表是通过对行或列的不同组合来对源数据进行统计汇总(求和、计数、求平均等汇总运算)。
    - pd.pivot_table(data, values, index, columns, aggfunc, fill_value, margins, dropna, margins_name, ...)

分类汇总

分类汇总

分类方法：

```
df.groupby(by, axis, as_index,sort,……    )
```

参数	说明
by	指定分类依据
axis	分类轴，0 或 'index'（默认）表示按行分类，1 或 'columns' 表示按列分类
as_index	布尔值，默认为 True，表示使用分类标签作为索引
sort	是否对分类键进行排序

分类汇总

方式1：

df.groupby(分类字段）[汇总字段].汇总函数()

如：计算各公司总销售额

分类字段　汇总字段

```
shopSale=salesData.groupby('公司')['销售额'].sum()
```

汇总函数

返回Series对象，分类字段是Series对象的索引

```
公司
京东        886543
天猫        515871
拼多多      258077
苏宁易购    386806
Name: 销售额, dtype: int64
```

分类汇总

方式2：

df.groupby(分类字段）.agg({“字段1”：汇总函数，“字段2”:“汇总函数”，……})

如：计算各公司总销售额

分类字段　　汇总字段

```
shopSale=salesData.groupby('公司').agg({'销售额':'sum'})
```

汇总函数

返回DataFrame对象，分类字段是DataFrame对象的行索引

公司	销售额
京东	886543
天猫	515871
拼多多	258077
苏宁易购	386806

分类汇总

如：计算各公司总销售额以及平均销售量

分类字段

汇总字段

```
shopSale1=salesData.groupby('公司').agg({'销售额':'sum','销售量':'mean'})
```

汇总函数

结果DataFrame对象的行索引

	销售额	销售量
公司		
京东	886543	15.70
天猫	515871	21.50
拼多多	258077	5.75
苏宁易购	386806	14.80

```
shopSale2=salesData.groupby('公司',as_index=False).agg({'销售额':'sum','销售量':'mean'})
```

结果DataFrame对象的一列

	公司	销售额	销售量
0	京东	886543	15.70
1	天猫	515871	21.50
2	拼多多	258077	5.75
3	苏宁易购	386806	14.80

分类汇总

如：计算各公司各类商品的总销售额

```
shopSale3=salesData.groupby(by=['公司','商品名称'])['销售额'].sum()
```

或

```
shopSale4=salesData.groupby(by=['公司','商品名称']).agg({'销售额':'sum'})
```

公司	商品名称	
京东	Apple Macbook Air 13.3	107982
	Apple iPhone 11	59990
	Apple iPhone XR	81584
	DELLXPS13.3英寸	187980
	三星(SAMSUNG) 500GB 固态硬盘	13380
	佳能（Canon）EOS 5D Mark IV	81495
	佳能（Canon）EOS 80D	82990
	华为 HUAWEI Mate 30	77382
	华为笔记本 MateBook 14	125378
	小米9 Pro	68382
天猫	Apple iPhone 11	179970
	Apple iPhone XR	203960
	佳能（Canon）EOS 800D	24995
	尼康（Nikon）D850	73196
	西部数据(WD)2TB	9780
	西部数据(WD)4TB	23970
拼多多	Apple Macbook Pro 13.3	64194
	佳能（Canon）EOS 5D Mark IV	48897
	佳能（Canon）EOS 80D	49794
	尼康（Nikon）D750	95192
苏宁易购	ThinkPad T480	131988
	ThinkPad X1 Carbon	179988
	华为 HUAWEI Mate 30 Pro	50392
	希捷(Seagate)2TB	14970
	希捷(Seagate)4TB	9468

Name: 销售额, dtype: int64

		销售额
公司	商品名称	
京东	Apple Macbook Air 13.3	107982
	Apple iPhone 11	59990
	Apple iPhone XR	81584
	DELLXPS13.3英寸	187980
	三星(SAMSUNG) 500GB 固态硬盘	13380
	佳能（Canon）EOS 5D Mark IV	81495
	佳能（Canon）EOS 80D	82990
	华为 HUAWEI Mate 30	77382
	华为笔记本 MateBook 14	125378
	小米9 Pro	68382
天猫	Apple iPhone 11	179970
	Apple iPhone XR	203960
	佳能（Canon）EOS 800D	24995
	尼康（Nikon）D850	73196
	西部数据(WD)2TB	9780
	西部数据(WD)4TB	23970
拼多多	Apple Macbook Pro 13.3	64194
	佳能（Canon）EOS 5D Mark IV	48897
	佳能（Canon）EOS 80D	49794
	尼康（Nikon）D750	95192
苏宁易购	ThinkPad T480	131988
	ThinkPad X1 Carbon	179988
	华为 HUAWEI Mate 30 Pro	50392
	希捷(Seagate)2TB	14970
	希捷(Seagate)4TB	9468

PART 06

数据可视化

数据类型

定类型

- 不同的类别
- 例子：水果、性别、电影类型等等

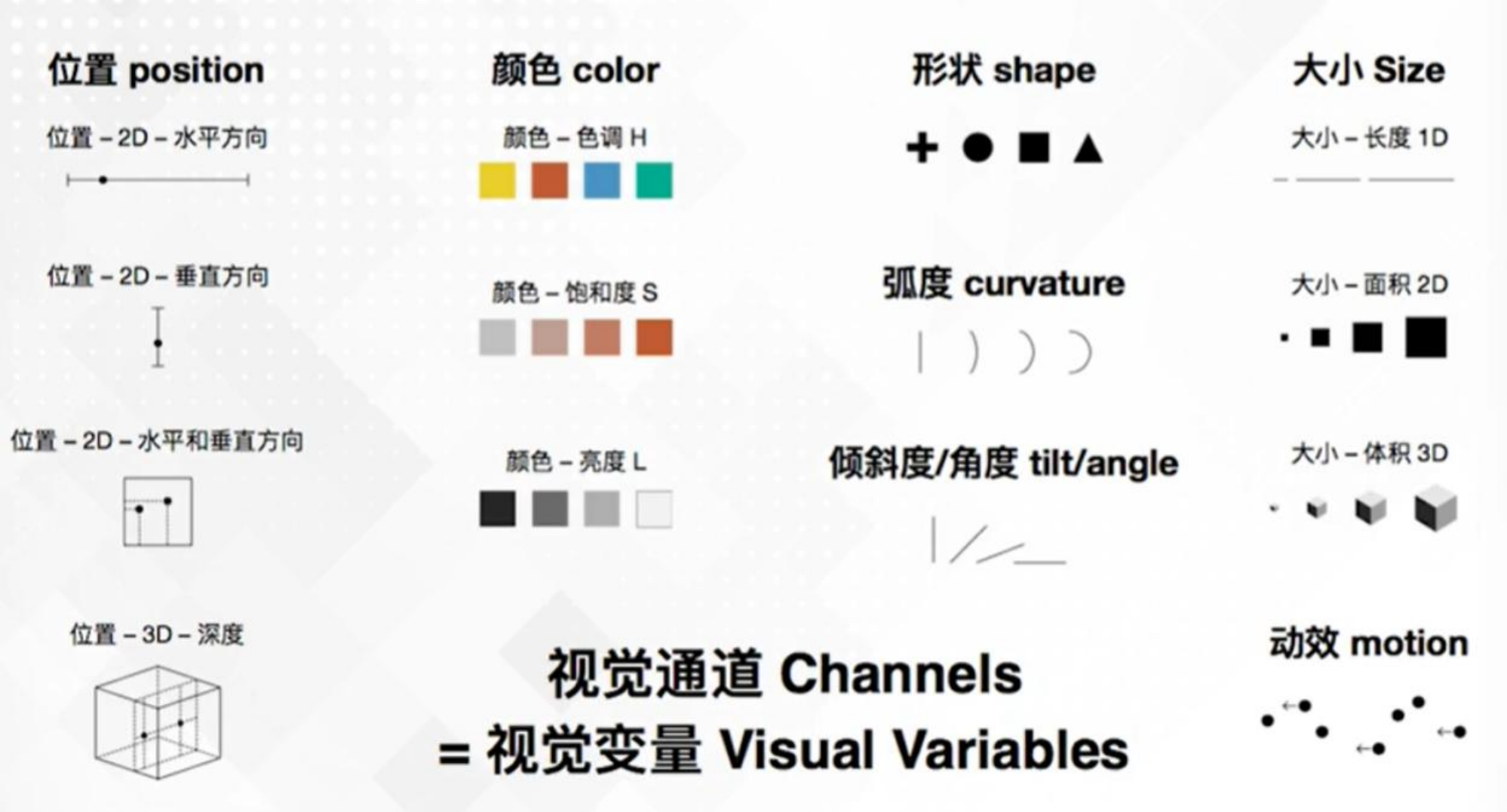
有序型

- 定序型（ordinal）：可比较，如排名、序号
- 定量型（quantitative）：有数值，如长度、重量、数量

符号和通道

- 符号标记
  - 基本可视化图形元素
  - 表示数据项和连接
- 视觉通道
  - 符号标记的表现形式
  - 表现数据的属性

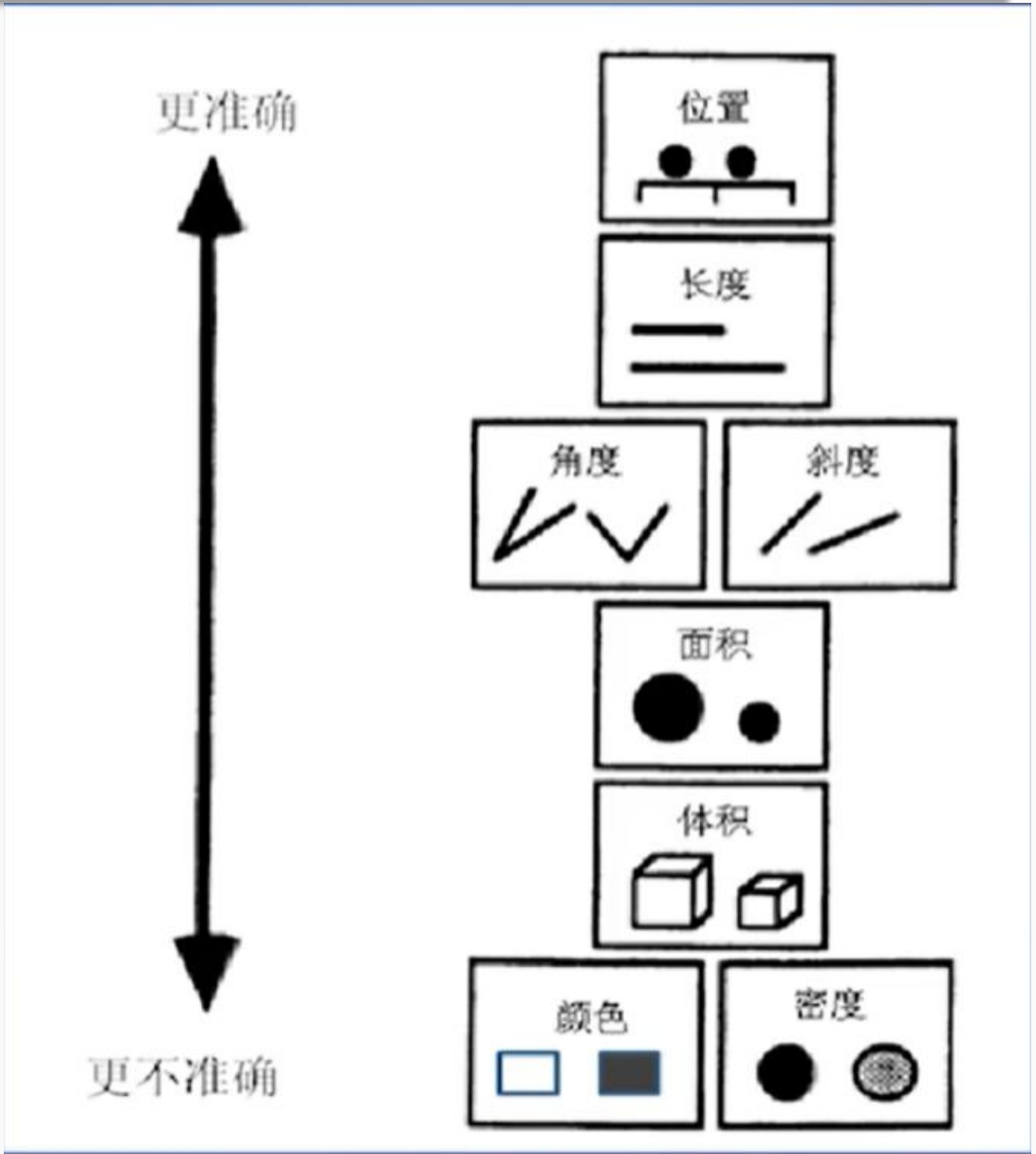
视觉通道


位置 position
位置 – 2D – 水平方向
位置 – 2D – 垂直方向
位置 – 2D – 水平和垂直方向
位置 – 3D – 深度
颜色 color
颜色 – 色调 H
颜色 – 饱和度 S
颜色 – 亮度 L
形状 shape
弧度 curvature
倾斜度/角度 tilt/angle
大小 Size
大小 – 长度 1D
大小 – 面积 2D
大小 – 体积 3D
动效 motion
视觉通道 Channels
= 视觉变量 Visual Variables

数据类型的可视化映射

数值型
有序型
类别型
位置
长度
角度
斜度
面积
体积
密度
饱和度
色调
纹理
连接关系
包含关系
形状
位置
密度
饱和度
色调
纹理
连接关系
包含关系
长度
角度
斜度
面积
体积
形状
位置
色调
纹理
连接关系
包含关系
密度
饱和度
形状
长度
角度
斜度
面积
体积
min
max
S
U

可视编码元素的优先级



可视化工具

- 免费
  - Python 系 (Matplotlib, Seaborn, Plotly, Pyecharts): 全部免费开源。可以随意集成到任何教学程序或商业软件中。
  - JS 系 (ECharts, D3.js): 全部免费开源。尤其 ECharts，在网页展示和跨平台适配上非常友好。
  - PandasAI:核心代码开源（免费），但如果调用闭源大模型（如 GPT-4 或 Gemini）的 API，需要支付模型 token 费。
- 有条件免费
  - Power BI
  - Tableau
  - FineBI
- 大模型出图

  输入数据快速出图，但有可能图表并不是真实反映数据。

不同图表的适用场景

http://tuzhidian.com

不同图表的适用场景

https://datavizcatalogue.com/ZH/index.html

数据可视化

➢数据分析中常用于数据可视化的Python包如下：

- ◆matplotlib
- ◆seaborn
- ◆pandas
- ◆pyecharts
- ◆plotly
- ◆……

pyecharts特点

- 与Echarts结合

  （echarts：https://echarts.apache.org/zh/index.html）
- 与web端结合
- 超强交互性
- 操作简单
- 可轻松集成至 Flask，Django 等主流 Web 框架

pyecharts安装

- Windows系统
  - pip install pyecharts
- Mac系统
  - pip3 install pyecharts

图表绘制流程

#导包

from pyecharts.charts import 图表类型

from pyecharts import options as opts

#画图

1. #初始化型图表类型 chart_name=图表类型()
2. 添加数据
   - chart_name.add_xaxis() #添加横坐标信息
   - chart_name.add_yaxis() #添加纵坐标数据
3. 全局配置参数 .set_global_opts()
4. 系列配置参数 .set_series_opts()
5. chart_name.render() #生成html文件

图表绘制

- 案例展示中找效果： https://gallery.pyecharts.org/#/
- 语法帮助中查参数设置

https://pyecharts.org/#/zh-cn/quickstart

基本绘图


```
from pyecharts.charts import Bar
from pyecharts import options as opts
bar = (
    Bar()#图表函数
    .add_xaxis(["衬衫", "羊毛衫", "雪纺衫", "裤子",
"袜子"]) #x轴数据
    .add_yaxis("商家A", [5, 20, 36, 10, 75, 90])#
    .set_global_opts(title_opts=opts.TitleOpts(ti
售数据", subtitle="服装类")) #全局配置（标题、副标题）
    .set_series_opts(
        label_opts=opts.LabelOpts(is_show=False),
        markline_opts=opts.MarkLineOpts(
            data=[opts.MarkLineItem(y=
        )#系列配置（标签、标记线），
    )
)
bar.render("柱形图.html")
```

根据要绘制的图形，导入不同的图形库
Bar, Pie, Scatter, ……

数据必须是**列表**类型的

生成HTML文档，无参数时，在当前目录下生成render.html文件，有参数时，根据参数生成指定路径和文件名的文件。

设置图表

- 使用 options 配置项，在 pyecharts 中，一切皆 Options。

```
from pyecharts import options as opts
```

- 全局配置
  - https://pyecharts.org/#/zh-cn/global_options

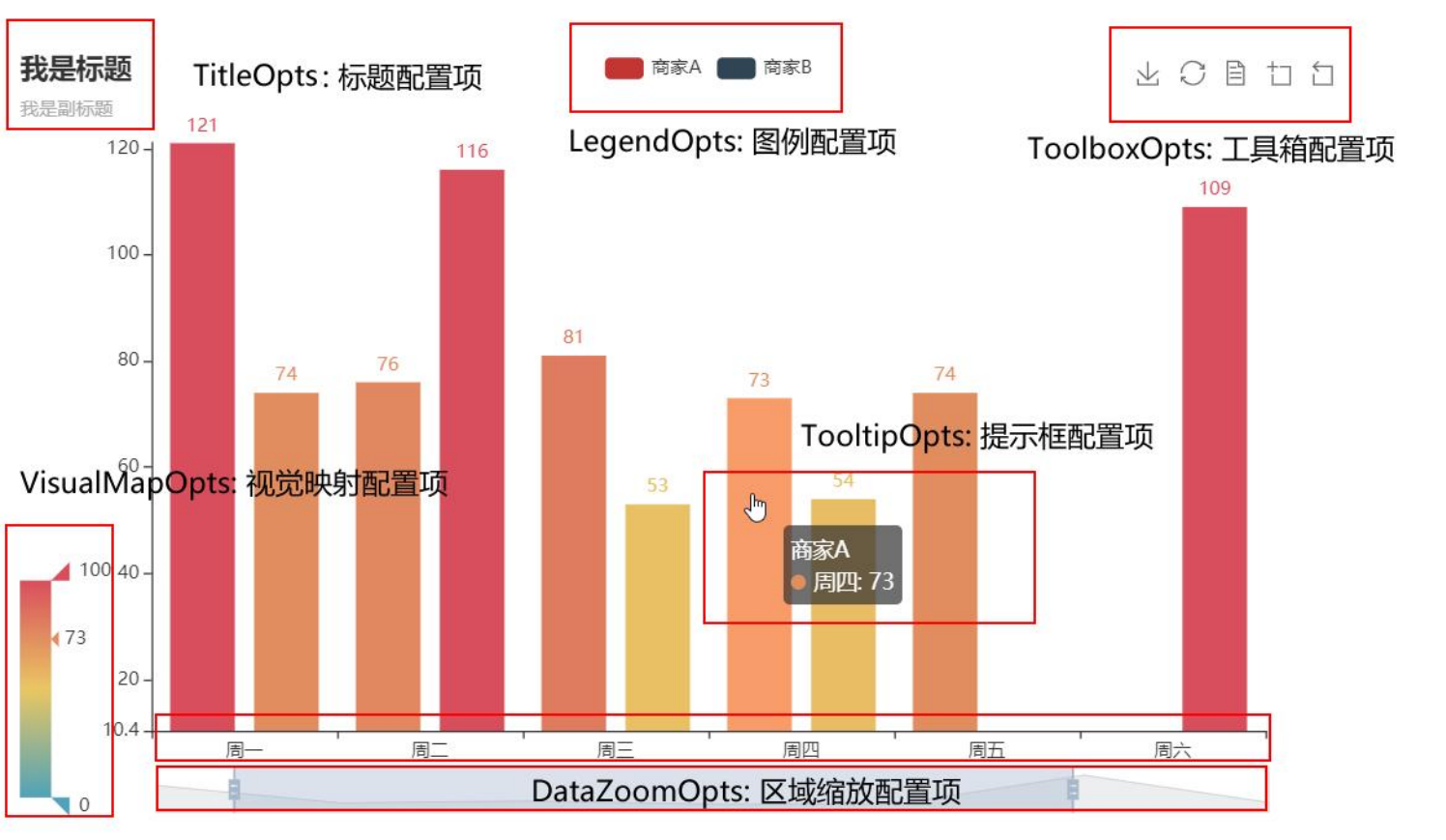
```
.set_global_opts(title_opts=opts.TitleOpts(title="主标题",subtitle="副标题"))
```

- 系列配置
  - https://pyecharts.org/#/zh-cn/series_options

```
.set_series_opts(****)
```

设置图表

➢全局配置


我是标题
我是副标题
TitleOpts：标题配置项
商家A
商家B
LegendOpts: 图例配置项
ToolboxOpts: 工具箱配置项
VisualMapOpts: 视觉映射配置项
TooltipOpts: 提示框配置项
商家A
周四: 73
DataZoomOpts: 区域缩放配置项
121
74
76
116
81
53
73
54
74
109
周一
周二
周三
周四
周五
周六

全局配置


```
from pyecharts.charts import Bar #柱形图
from pyecharts import options as opts
from pyecharts.globals import ThemeType
#设置主题
from pyecharts.faker import Faker

bar=Bar(init_opts=opts.InitOpts(theme=Th
emeType.ESSOS))
bar.add_xaxis(Faker.choose())
bar.add_yaxis("商家
A",Faker.values(),stack="stack1")
bar.set_global_opts(title_opts=opts.Titl
eOpts(title="柱形图",subtitle="此表数据为
虚构数据"))
bar.render()
```

系列配置

➢标记点、标记线

```
……

bar.set_series_opts(label_opts=opts.LabelOpts(is_show=False),
                    markpoint_opts=opts.MarkPointOpts(
                    data=[
                        opts.MarkPointItem(type_="max",name="最大值"),
                        opts.MarkPointItem(type_="min",name="最小值
")
                    ]),
                    markline_opts=opts.MarkLineOpts(

 data=[opts.MarkLineItem(type_="average",name="平均值")]

                    )
                    )
```

选择主题


```
from pyecharts.globals import ThemeType #设置主题
```

```
Bar(init_opts=opts.InitOpts(theme=ThemeType.LIGHT))
```

theme的取值:
https://pyecharts.org/#/zh-cn/themes

多组数据

➢用add_yaxis加载多组数据

```
bar.add_yaxis("商家A",Faker.values(),stack="stack1")
bar.add_yaxis("商家B",Faker.values(),stack="stack1")
bar.add_yaxis("商家C",Faker.values(),stack="stack2")
bar.add_yaxis("商家D",Faker.values(),stack="stack2")
```

➢ stack属性用来控制是否是堆叠图形，如果是多个系列堆叠在一起，需将stack指定为相同的值。

x轴和y轴交换


```
bar.reversal_axis()
```

带有滑块的图表

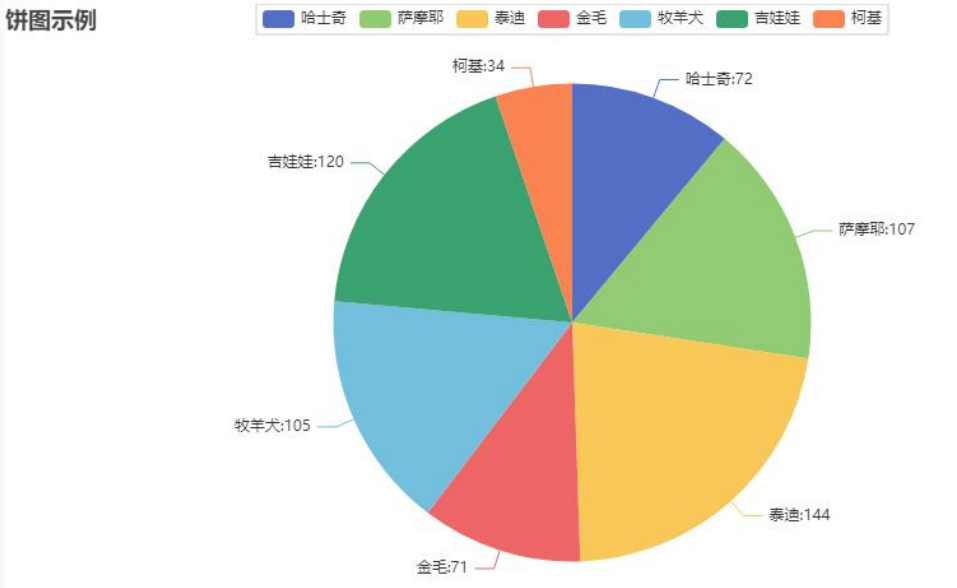
➢添加全局配置

```
datazoom_opts=[opts.DataZoomOpts()]
```

饼图

➢系列数据项：[(key1,value1),(key2,value2),……]

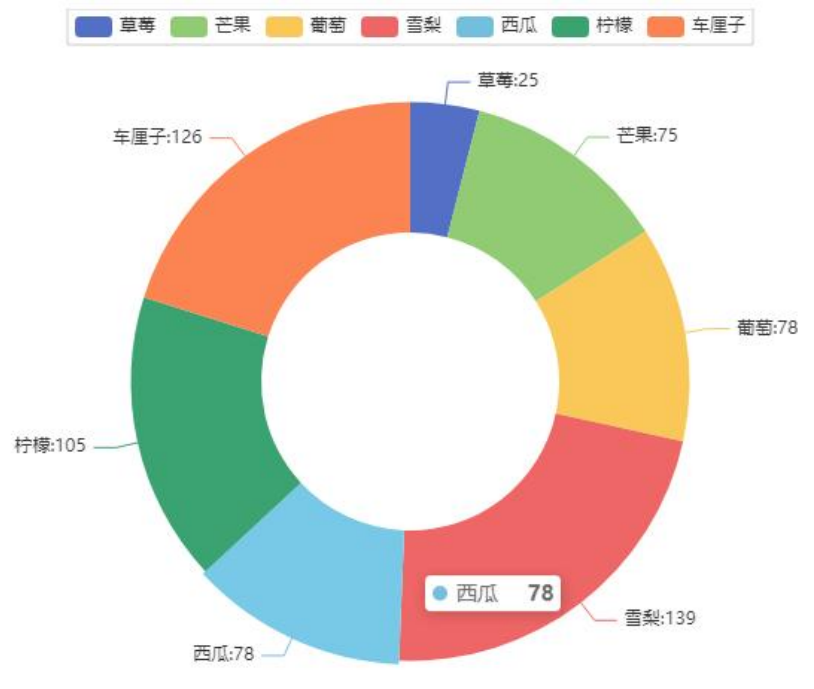
```
from pyecharts.charts import Pie
……
pie=Pie()
pie.add("",[list(z) for z in
zip(Faker.choose(),Faker.values
())])
pie.set_global_opts(title_opts=opt
s.TitleOpts(title="饼图示例"))
pie.set_series_opts(label_opts=opt
s.LabelOpts(formatter="{b}:{c}"))
……
```



环状图


```
pie.add("",
        [list(z) for z in zip(Faker.choose(),Faker.values())
        radius=["40%","75%"])
```

radius：以列表形式给出
内环半径和外环半径



wordcloud词云图

➢数据格式

data=[(词1,词频),(词2，词频),……]

data = [("生活资源", "999"), ("供热管理", "888"), ("供气质量", "777"), ("生活用水管理", "688"), ("一次供水问题", "588"), ……]

- 当指定mask_image时，参数可以是 j[“base64”]，也可以是图像文件路径字符串，目前支持 jpg, jpeg, png, ico 的格式，其他的图片格式待测试)
- 如果使用了 mask_image 之后第一次渲染会出现空白的情况，再刷新一次就可以了（Echarts 的问题）

wordcloud词云图

➢词云图中词的准备

```
import jieba
import collections
with open("data/report20CH.txt", "r", encoding="utf-8") as f:
    data = f.read()
words_cut = jieba.cut(data)#分词
words_counter=collections.Counter(words_cut)#统计词频
words=list(words_counter.items())#将词及词频转为列表
```

➢思考：去掉停用词制作词云图

保存为图片

➢make_snapshot 用于 pyecharts 直接生成图片

```
#保存为图片
from pyecharts.charts import Bar
from pyecharts.render import make_snapshot
# 使用 snapshot-selenium 渲染图片
from snapshot_selenium import snapshot

bar = (
    Bar()
    .add_xaxis(["衬衫", "羊毛衫", "雪纺衫", "裤子",
"高跟鞋", "袜子"])
    .add_yaxis("商家A", [5, 20, 36, 10, 75, 90])
)
make_snapshot(snapshot, bar.render(), "bar.png")
```

综合案例

- 利用pyecharts实现可视化大屏
  - 1. 获取数据
    - 下载并整理数据 https://data.stats.gov.cn/easyquery.htm?cn=E0105
    - 整理数据，去掉表格前几行和后几行非数据清单的数据
  - 2.数据处理
    - 数据清洗 —— 去掉有空值的记录
    - 针对每种图表选取合适的数据
  - 3.绘制各子图
    - 柱形图
    - 饼图
    - 词云图
    - ……
  - 4.生成可视化大屏
    - 原理
      - 采用pyecharts库的page方法，组合图表，生成大屏
    - 方法
      - 1. 生成大屏网页，调整 网页中图表布局
      - 2. 点击“save config"保存配置，并将该文件放至当前工作目录下
      - 3. page.save_resize_html('初始网页.html',cfg_file='chart_config.json',dest='最终网页.html')

综合案例——实现大屏可视化

1.绘制各子图表

- 可以每个图表定义一个函数

```
def bar_city_counts():
    bar=(
        Bar()
        .add_xaxis(city_counts.index.tolist())
        .add_yaxis("景点个数", city_counts.values.tolist())
        .set_global_opts(
            title_opts=opts.TitleOpts(title="4A级以上景点个数"),
            datazoom_opts=opts.DataZoomOpts(),
        )
    )
    return bar
```

- 也可以将各子图绘制放在独立的python文件中，需要时import导入

2.创建一个Page对象

- 创建Page对象，并运用DraggablePageLayout布局。
- Page就相当于一张白纸，而DraggablePageLayout允许我们拖拽图表来布局

```
page = Page(layout=Page.DraggablePageLayout,page_title="旅游景点数据可视化")
```

综合案例

3. 用add()将图表添加到Page上。add()参数为chart对象

```
page.add(bar_city_counts(),
             bar_top20(),
             map_city_sales(),
             pie_price_counts(),
             scatter_price_sales(),
             bar_rating_counts(),
             wordcloud_intro())
```

4. 用render()方法，将Page保存为一个html文件。

```
page.render("初稿.html")
```

5. 打开“初稿.html”调整图表布局，并按save config进行保存。

综合案例

6. 用保存好的配置，重新渲染网页，渲染后的网页即是最终的大屏网页

```
from pyecharts.charts import Page
page=Page(layout=Page.DraggablePageLayout,page_title='全国出游信息')
page.save_resize_html("初稿.html",cfg_file="chart_config.json",dest="
全国出游信息大屏可视化.html")
```

PART 07

作品发布

作品发布

- 部署作品到网络服务器

谢谢！